

Streaming Singlish ASR for Telephony

Design document - approach, trade-offs, evaluation, and results

WER	CER	Compute RTF	Chunk
18.13%	11.58%	0.0736	560 ms

1. Problem and design goals

Objective. Adapt an ASR system to Singlish and telephony-style speech while retaining true streaming behaviour. The system must emit partial hypotheses as audio arrives, maintain state across the utterance, and remain suitable for a conversational agent. Repeatedly calling an offline model on isolated fixed chunks is not a valid solution because context is reset at every call.

- Use a model with native cache-aware streaming inference and incremental RNNT decoding.
- Improve recognition of Singlish pronunciation, discourse markers, conversational phrasing, and code-switching patterns.
- Protect general English performance by mixing replay data into domain adaptation.
- Separate full-set accuracy evaluation from batch-size-1 conversational latency profiling.
- Provide a runnable demo showing partial/final text, WER/CER, coloured errors, and latency metrics.

Outcome. The $\leq 15\%$ overall WER target was not reached; the final normalized WER was 18.13%. The system did meet the core streaming requirement and showed substantial real-time compute headroom.

2. System approach

2.1 Model and streaming path

NVIDIA Nemotron Speech Streaming EN 0.6B was selected because the architecture natively supports cache-aware RNNT streaming, configurable chunk sizes, persistent encoder/decoder state, and partial hypotheses. Architectural fit was prioritised over using a stronger offline model that would only simulate streaming.

16 kHz mono audio -> ordered chunks -> cache-aware encoder/RNNT

-> partial hypotheses -> final hypothesis

State preservation. A streaming session retains the encoder channel cache, time cache, cache lengths, previous hypotheses, and previous prediction outputs. Every chunk advances the same utterance; the model is not restarted between chunks.

2.2 Data preparation and fine-tuning

The adaptation corpus combined curated Singlish speech with English replay data to reduce catastrophic forgetting. Data preparation included:

- transcript normalization and consistency checks;
- Whisper-based sanity checks using normalized WER/CER signals;
- removal of likely silence, non-speech, hallucinated, or severely mismatched examples;
- decontamination against the provided evaluation set;
- clean 16 kHz data and telephony-processed variants produced through a GSM-style channel;
- duration, manifest, and audio-format validation before training.

The final checkpoint was exported to a `.nemo` model and evaluated with the same cache-aware inference path used by the demo. Exact optimisation hyperparameters are intentionally omitted here because the submission focuses on the system methodology and measured outcomes.

2.3 Demo architecture

Component	Responsibility
FastAPI WebSocket backend	Loads base/fine-tuned models, owns streaming state, processes ordered chunks, and returns text plus timing events.
Gradio frontend	Uploads WAVs, selects model/chunk size, shows partial text, calculates metrics, colours word errors, and exports traces.
Real-time mode	Follows the audio clock for a user-facing live demonstration.
Fast evaluation mode	Removes sleep between chunks while preserving order and caches; it remains streaming rather than batch ASR.

3. Evaluation methodology

3.1 Accuracy protocol

Evaluation set. The headline result uses a filtered/decontaminated manifest of 346 files (2.443 hours) across Tier 0, Tier 1, and Tier 2. Audio was evaluated as mono 16 kHz input with a 560 ms chunk and attention context [70, 6].

Text normalization. References and hypotheses use Unicode NFKC normalization, case folding, markup/apostrophe/punctuation removal, preservation of letters and numbers, and whitespace collapse. WER and CER are calculated with jiwer.

Batching. Full-set accuracy used an effective batch size of 64 for efficient evaluation, while each utterance still advanced through ordered chunks with its own caches. Batch throughput is not reported as conversational latency.

3.2 Latency protocol

Latency was profiled separately with batch size 1 on 30 utterances. The profile reports model-compute RTF, end-to-end RTF, chunk compute percentiles, first non-empty partial latency, and final tail after the audio ends. This avoids presenting batch-64 throughput as a live-agent result.

4. Results

4.1 Accuracy

Group	Files	Normalized WER	Normalized CER
Overall	346	18.13%	11.58%
Tier 0	58	16.93%	10.26%
Tier 1	143	16.76%	10.65%
Tier 2	145	19.87%	12.93%

Tier 2 remained the most difficult group. The result should be described as a functioning streaming adaptation with measurable domain performance, not as achievement of the requested accuracy threshold.

4.2 Single-stream latency

Metric	Result
Compute RTF	0.0736
End-to-end RTF	0.0769
Chunk compute p50 / p95	40.59 / 45.49 ms
First partial p50 / p95	1161.63 / 1725.06 ms
Finalization p50 / p95	301.44 / 596.27 ms
Chunks over 560 ms compute budget	0%

The end-to-end RTF of 0.0769 indicates that the profiled inference path consumes far less compute time than the incoming audio duration. First-partial latency is larger than per-step compute because it also includes the amount of acoustic evidence required before a non-empty hypothesis becomes available.

5. Trade-offs and design decisions

Chunk size. Smaller chunks can update the interface more frequently but increase model steps, cache operations, and network/UI events. Larger chunks reduce overhead but delay the first opportunity to emit text. 560 ms was selected as the primary stable and efficient reporting point; the demo also exposes 80, 160, and 1120 ms.

Domain adaptation versus English retention. More Singlish-focused training may improve the harder tiers but can degrade general English. English replay was mixed into adaptation, but the remaining gap suggests that a better replay balance or staged continuation schedule is preferable to a more aggressive Singlish-only run.

Accuracy evaluation versus live serving. Batching is useful for full-set evaluation but can hide deployment latency. Accuracy is therefore reported over all 346 files, while latency is taken from batch-size-1 streams. The local Apple MPS demo is for interaction; headline metrics come from the controlled A100 evaluator.

Reproducibility versus repository size. The public repository contains the runnable code and aggregate outputs, but excludes task audio, per-sample transcripts, and the fine-tuned checkpoint because of access restrictions, licensing, privacy, and size. The README documents local checkpoint placement and supports a recorded walkthrough.

6. Limitations and next steps

- Overall WER remains above 15%, with Tier 2 the largest accuracy gap.
- Use more balanced English replay to protect Tier 0 while retaining Singlish gains.
- Add high-quality Tier 2 coverage and targeted transcript cleanup.
- Continue from the strongest checkpoint with a lower learning rate and explicit forgetting checks.
- Tune streaming decoding and endpointing, including partial stability across chunk sizes.
- Evaluate native 8 kHz telephony capture, packet loss, codec variation, noise, concurrency, memory, and quantisation.

Conclusion. The current system is a valid cache-aware streaming implementation with strong compute latency, a transparent evaluation methodology, and a practical live demo. The remaining challenge is accuracy rather than streaming feasibility.